

ADT Stack • container in which access is restricted to the TOP. (one end of the container) (LIFO) **NO ITERATOR!**

→ Dynamic Array (DS):

Stack:

- capacity: Int
- nrElem: Int
- elems: TElem[]

• init(s) → $\Theta(1)$

• destroy(s)

• push(s, e) → BC: $\Theta(1)$; WC: $\Theta(n)$ resize tail → $\Theta(1)$ am.

• pop(s) → $\Theta(1)$ → return.

• top(s) → $\Theta(1)$ remove + return

• isEmpty(s) → $\Theta(1)$

• size(s) → $\Theta(1)$

Array-Based: TOP at the end.
 SLL-Based: TOP at the head
 DLL-Based: TOP at either head or tail

ADT Queue • container in which access is restricted to the FRONT and REAR. (FIFO)

→ Circular Array: (convention) (*)

Queue: nrElem: Int
 capacity: Int
 elems: TElem[]
 front: Int
 rear: Int

• init(q) → $\Theta(1)$

• destroy(q)

• push(q, e) → BC: $\Theta(1)$; WC: $\Theta(n)$ → $\Theta(1)$ am. **push = insert at rear, rear increments**

• pop(q) → $\Theta(1)$ **pop = remove from front, front increments**

• top(q) → front of q → $\Theta(1)$

• isEmpty(s) → $\Theta(1)$

• front(q) = top(q) (DLL does not care)

where to place FRONT and REAR w an ARRAY?

⇒ ⇒ good if we push more

⇒ ⇒ good if we pop more

+ isFull() needed!

→ Dynamic Array same repres, but we can resize()

ADT PriorityQueue • container in which access is restricted to FRONT, REAR and each elem. has an associated priority. (TPriority), we can access ONLY ELEM WITH THE HIGHEST PRIORITY! (HPF) **NO ITERATOR!**

→ Dyn Array:

TElem PQ

- elem: TElem
- priority: TPriority

PriorityQueue

- capacity: Int
- nrElem: Int
- elems: TElem PQ[]
- relation: TRelation

• init(pq, R) ✓

• destroy(pq) ✓

• push(pq)

• pop(pq)

• top(pq)

• isEmpty(pq)

LINKED LIST (DS) • linear DS in which the order of elems is det. not by indexes, but by a pointer which is placed in each elem.

• NODE = data + pointer to the address of next node (+/- prev)

ADT Degree • we can insert + delete from both ends.

Degree: (CircularArray)

- cap: Int
- nrElem: Int
- elems: TElem[]
- front: Int
- rear: Int

• same op. as

• init(), destroy(), pushFront(e) → $\Theta(1)$ am. pushBack(e) → $\Theta(1)$ am. popFront(), isEmpty(), popBack(), front(), back()

SLL:

SLLNode:

- info: TElem
- next: TSLLNode

SLL:

- head: TSLLNode

• search, add to ≠ places, delete from ≠ places, get from pos

• search(sel, elem) → $\Theta(m)$ (pay attention what you return)

• insertFirst(sll, elem) → $\Theta(1)$

• insertAfter(sll, current, elem) → $\Theta(1)$

• insertPos(sll, pos, elem) → $\Theta(m)$

DLL:

DLLNode:

- info: TElem
- next: TDLLNode
- prev: TDLLNode

DLL:

- head: TDLLNode
- tail: TDLLNode

• same op. as SLL

• insertLast(dll, elem)

• insertPos(dll, pos, elem)

SLL (sorted SLL)

SSLLNode:

- info: TComp
- next: TSLLNode

SSLL

- head: TSLLNode
- rel: TRelation

• init, insert, search (stop earlier), delete, getFromPos, iterator → like SLL

SLCircularList • each node has a successor

“last node” = the node whose next is the head of the list

• retain last node to help insertion

CSLLNode:

- info: TElem
- next: TCSLLNode

CSLL:

- head: TCSLLNode

• insertFirst($\Theta(m)$)

• deleteLast → $\Theta(m)$

• iterator → invalid when the next of currentElem is head.

⊕ boolean flag

Skiplist (DS) • fast SEARCH in an ordered LL.

Node:

- info: TElem
- nextToLevel: Node

Skiplist:

- head: Node
- maxlev: Int
- rel: Relation

! Search: start from top level, move right while next < elem, if cannot move right, go down one lev. ⇒ WC: $\Theta(m)$ ⊕ extra space for lev.

SLLA (no pointers)

• allows to add/insert anywhere as long as links are not correctly.

SLLA: (two arrays)

- elems: TElem[] (elems)
- next: Integer[] (links)
- cap: Integer (same)
- head: Integer (index)
- firstEmpty: Integer (index)

• same op. as SLL.

SLLAIterator: ⇒ one array for elems, one for next

- slla: SLLA
- current: Integer

DLLA (no pointers)

DLLANode:

- info: TElem
- next: Integer
- prev: Integer

DLLA (one array)

- nodes: DLLANode[]
- cap: Integer
- head: Integer
- tail: Integer
- firstEmpty: Integer
- size: Integer

• allocate(), free() to make it closer to a LL w Dyn. Alloc.

ADT Priority Queue

→ Dyn. Array / LL → decide how we store elems in the array/list:

- I. ORDERED BY PRIORITY or
- II. ORDERED BY INSERTION

XOR Linkedlist

• link = prev XOR next

XORNode:

- info: TElem
- link: Address TInt

XORList:

- head: XORNode
- tail: XORNode

PRINT ANY CONTAINER:

subalgorithm print(c) is:

```

while valid(it) execute
elem ← getcurrent(it)
print elem
next(it)
  
```

1

ADT Degree

→ dynamic array of fixed-size arrays

3D Map

6D Map

K-Tree

V-Tree

ADT Priority Queue

→ Dyn. Array / LL → decide how we store elems in the array/list:

- I. ORDERED BY PRIORITY or
- II. ORDERED BY INSERTION

ADT Priority Queue

→ Dyn. Array / LL → decide how we store elems in the array/list:

- I. ORDERED BY PRIORITY or
- II. ORDERED BY INSERTION

ADT Priority Queue

→ Dyn. Array / LL → decide how we store elems in the array/list:

- I. ORDERED BY PRIORITY or
- II. ORDERED BY INSERTION

ARRAY (DS)

- fixed nr. of elems
- elems of the same type
- contiguous memory block
- address of the first elem.
- ⊕ elems accessed in $\Theta(1)$ ✓
- ⊖ we don't always know from the start the nr. of elem.

DYNAMIC ARRAY

capacity: Integer
nrElem: Integer
elems: TElem[]

- size() $\rightarrow \Theta(1)$
- getElem(p) $\rightarrow \Theta(1)$
- setElem(p, e) $\rightarrow \Theta(1)$
- iterator() $\rightarrow \Theta(1)$
- addToEnd(e) $\rightarrow \Theta(1)$ am.
- addToPos(p, e) $\rightarrow \Theta(m)$
- deleteFromEnd() $\rightarrow \Theta(1)$
- deleteFromPos(p) $\rightarrow \Theta(m)$
- deleteGiven(e) $\rightarrow \Theta(m)$

ArrayIterator (not needed, we have positions!)

array: DynamicArray
current: Integer

- first() $\rightarrow \Theta(1)$
- next() $\rightarrow \Theta(1)$
- valid() $\rightarrow \Theta(1)$
- getCurrent() $\rightarrow \Theta(1)$

STATIC ARRAY

Array:
elems[CAPACITY]

- init(da, cap)
- resize(da)
- destroy(da)

ADT Bag: • no positions, no order, no uniqueness

→ Dynamic Array (DS):

Bag: R1 → more memory

capacity: Integer
nrElem: Integer
elems: TElem[]

- ✓ add(b, e) $\rightarrow \Theta(1)$ am (!)
- ✓ remove(b, e) $\rightarrow \Theta(m)$
- size(b) $\rightarrow \Theta(1)$
- ✓ search(b, e) $\rightarrow \Theta(m)$
- init(b)
- destroy(b)
- ✓ iterator(b, it)
- ✓ nrOccurrences(b, e) $\rightarrow \Theta(m)$
- isEmpty() $\rightarrow \Theta(1)$

Bag: R2 → less memory

capacity: Integer
nrElem: Integer
elems: TElem[]
freq: TElem[]

- nrOccurrences (easier)
- iterator (harder)

② Bag: R3

No positions $\Rightarrow$ more performance
Iterators $\Rightarrow$ faster even on lists (where we already have pos)

ADT List • only containers with POSITIONS • order, pos.

→ Dyn. Array:

List:
cap: Int
nrElem: Int
elems: TElem[]

Iterator

Position = Int (index)

- init() • search(l, e); isEmpty(l)
- first(), last(), size(l)
- valid(l, p) • addBefore(l, e)
- next(l, p) • addAfter(l, e)
- prev(l, p) • addBefore(l, p)
- getElem(l, p) • addBefore(l, p)
- position(l, e) • remove(l, p)
- setElem(l, p, e) • remove(l, e)

(1, A), (1, B), (1, A), (1, C)

MultiMap (R1) (*)

MultiMap (R2):

(1, {A, B, C}) = elemsTo

TElem:

Key: Tkey

VL: ValuesList

MultiMap: (R2)

capacity: Int

nrElem: Int

elems: TElem[]

ADT IndexedList • list

in which TPosition = Int

$\Rightarrow$ first, last, next, prev, valid.

• addBefore(l, i, e)

ADT IteratedList • list

in which TPosition =

= Iterator $\Rightarrow$ next, prev, valid

• addBefore(l, it, e)

ADT SortedList • one

add op., $\neq$ setElem()

Indexed/Iterated

ADT SortedBag

• Bag + relation passed to the init() function

SortedBag: R1

capacity: Int

nrElem: Int

elems: TElem[]

Relation R

- add must preserve sorting by R
- iterator returns in sorted order by R

• init(b, R)

SortedBag: R2

ADT Map (dictionary) • no positions, no order, unique keys, store (key, value) pairs, values may not be unique

(*) Map: → add checks if elem already exists (key)

TElem: (K, V)

Key: Tkey

value: TValue

• init(m)

• destroy(m)

• add(m, k, v) "RT"

• remove(m, k)

• search(m, k)

• iterator(m, it)

• size(m)

• isEmpty(m)

• keys(m, s) SET

• values(m, b) BAG

• pairs(m, s) SET

ADT MultiMap • a key way

have more values (unique values or not)

ADT SortedMultiMap

• init, iterator

• add checks rel R

1 $\rightarrow$ A; 1 $\rightarrow$ A; 1 $\rightarrow$ A is valid ✓

1 $\rightarrow$ A; 1 $\rightarrow$ B; 1 $\rightarrow$ C is valid ✓

(add does not check key uniqueness)

Matrix with compressed sparse column repres:

Matrix:

Lines: Integer[]

Cols: Integer[]

Values: TElem[]

capacity: Integer

nrElem: Integer

nrLines: Integer

nrCols: Integer

→ Values[i] = (lines[i], column)

→ Cols[j] = pos in values where column j starts.

• same operations as Matrix

Matrix with compressed sparse line representation:

→ values[i] = (line, Cols[i])

→ Lines[i] = pos in values where row i starts.

ADT Set • no positions, no order, unique.

→ Dynamic Array (DS):

Set:

capacity: Int

nrElem: Int

elems: TElem[]

• init(s)

• add(s, e)

• remove(s, e)

• search(s, e)

• size(s)

• isEmpty()

• iterator(s, it)

• destroy(s)

• Bag + add(e) first checks search(e) and inserts only if elem. e is not already there

ADT SortedSet • Set + relation

passed to the init() function

• add must preserve sorting by R

• iterator returns in sorted order

SortedSet:

capacity: Int

nrElem: Int

elems: TElem[]

Relation R

ADT Matrix • unique positions determined by line, col.

= TWO-DIMENSIONAL ARRAY.

Matrix: → more memory for sparse matrices

nrLines: Int

nrCols: Int

elems: TElem[]

• init(mat, nrLines, nrCols)

• nrLines(mat) $\rightarrow \Theta(1)$

• nrCols(mat) $\rightarrow \Theta(1)$

• element(mat, i, j) $\rightarrow \Theta(1)$

• modify(mat, i, j, val) $\rightarrow \Theta(1)$

Access formula:

pos = 1 * nrCol + j

Iterators $\rightarrow$ useful for iterating through the elements of any container in an uniform way

ADT SortedMap • Map + order

given by a relation which is passed to the init() fct.

SortedMap:

capacity: Int

nrElem: Integer

elems: TElem[]

relation: Relation

• add preserves rel R

• iterator returns in sorted order.

obtained from Cols

↑

Matrix

Lines: Integer[]

Cols: Integer[]

Values: TElem[]

capacity: Integer

nrElem: Integer

nrLines: Integer

nrCols: Integer

→ Values[i] = (line, Cols[i])

→ Lines[i] = pos in values where row i starts.

• same operations as Matrix

Matrix with compressed sparse line representation:

→ values[i] = (line, Cols[i])

→ Lines[i] = pos in values where row i starts.

ADT Set • no positions, no order, unique.

→ Dynamic Array (DS):

Set:

capacity: Int

nrElem: Int

elems: TElem[]

• init(s)

• add(s, e)

• remove(s, e)

• search(s, e)

• size(s)

• isEmpty()

• iterator(s, it)

• destroy(s)

• Bag + add(e) first checks search(e) and inserts only if elem. e is not already there

ADT SortedSet • Set + relation

passed to the init() function

• add must preserve sorting by R

• iterator returns in sorted order

SortedSet:

capacity: Int

nrElem: Int

elems: TElem[]

Relation R

Priority Queue (emergency room)

- each elem has a priority (TPriority)
- only elem with highest priority is accessible • FIFO
- HPF
- relation R
- operations:
 - 1) init (pg, R)
 - 2) destroy (pg)
 - 3) push (pg, e, p)
 - 4) pop (pg)
 - 5) top (pg)
 - 6) isEmpty (pg)
- no iterator!

K-ary tree

Node:

info: TElem
parent: TNode
leftChild: TNode
rightSibling: TNode

Traversals:

- 1) DFS (depth)
- 2) BFS (level-ori)

DFS → stack, visited
BFS → queue, visited

ADT Stack

- the access to the elements is restricted to one end of the container (TOP of the stack)
- LIFO
- For Dynamic Arrays, you should put the top of the stack towards the end of the array!
 - pos 1 → push/pop in $\Theta(m)$
 - end pos → push/pop in $\Theta(1)$
- ARRAYS - static / dynamic
- LINKED LISTS - SL / DL
- Special stack with getMinimum operation with $\Theta(1)$ time?
 - use 2 stacks.

BinomialNode

info: TElem
parent: BinomialNode
leftChild: BinomialNode
rightChild: BinomialNode
rightSibling: BinomialNode

BinomialTree

root: BinomialNode

- merge 2 heaps → $O(\log_2 m)$
- push (= create a bin. heap with only that node) → $O(\log_2 n)$ WC; $\Theta(1)$ amortized.
- top (min/max elem) = root (elem with highest priority) → $O(\log_2 m)$
- pop (rem. min) $O(\log_2 n)$

PriorityQueue Operation Compl.

Op	Sorted	Non-Sort	Heap
push	$O(m)$	$\Theta(1)$	$O(\log_2 m)$
pop	$\Theta(1)$	$\Theta(m)$	$O(\log_2 m)$
top	$\Theta(1)$	$\Theta(m)$	$\Theta(1)$

Binary Heap (DS) → efficient repres. for Priority Queues = dyn. array + binary tree

→ DA = levels

→ each node has 2 children except for the last 2 rows (left → right)

→ elem. on pos i ⇒ children are on pos 2i, 2i+1; ⇒ parent of i is $\lfloor i/2 \rfloor$; STRUCTURE = no gaps; PROP (min)

BinaryHeap

cap: Integer
len: Integer
elems: TElem[]
(rel: TRelation) → For Priority Queue

the only operations on a heap:

- push
- pop

Height of Heap = $\log_2 m$ levels = $1 + 2 + 4 + 8 + \dots = m$

- bubble-up → swap with parent until reached the correct position (prop.)
- bubble-down → swap with best child until leaf / respects prop.
- leaves = nodes with indices: $\lfloor m/2 \rfloor + 1, \dots, m$; add to heap = push to PriorityQueue; remove from heap = pop

Subalgorithm buildTree (postfix)

init(s)

for e in postfix

if e is an operand

node ← allocate()

[node].info ← e

[node].left ← NIL

[node].right ← NIL

push(s, node)

else

e1 ← pop(s); e2 ← pop(s)

node ← allocate()

[node].info ← e

[node].left ← e1

[node].right ← e2

push(s, node)

return root

HT not suitable for SORTED stuff, except the Sorted Map; HT = FAST SEARCH

ORDERED TREE: order of children is relevant

COMPLETE TREE: height = $\log_2 m$

HEIGHT OF A TREE (rec.)

height(node)

if node == null return -1

return 1 + max(height(node.left), height(node.right))

$O(m)$

degree = nr. children
leaf / internal n.

DEPTH: root
HEIGHT: longest
node

Linear probing: $h_1(k, i) = (h'(k) + i) \% m$

Double hashing: $h_2(k, i) = (h'(k) + i \cdot h''(k)) \% m$

$h'(k) = k \% m$ or other method (given); $h''(k)$;

BUILD BIN. TREE from arithmetic expression:

- operand (a) → push to stack; operator ⇒ pop 2 from stack ⇒ merge those trees (postfix as input)
- last thing popped goes on the RIGHT!

[node].left ← e2
[node].right ← e1
push(s, node)

TREE → conn., acyclic graph; one node = ROOT;

BINARY TREE

- ordered tree with max 2 children (left / right child)
- FULL when every internal node has 2 ch.
- COMPLETE when all leaves on the same level

BINARY SEARCH TREE

- BT + extra property: each elem left subtree ≤ current ≤ each right subtree.
- FOR SORTED CONTAINERS
- $O(\text{height})$
- insert: $O(m)$

TreeNode

info: TElem
parent: TTreeNode
left: TTreeNode
right: TTreeNode

BINOMIAL TREE

- order k → 2^k nodes
- order k ⇒ 2^k nodes ⇒ k children on the level after the root

ADT BinomialTree

Node

info: TElem
left: TNode
right: TNode

BT:

nodes: TNode[]
root: TNode
firstEmpty: TNode
cap: Int

BINOMIAL HEAP

- collection of binomial trees → at most $\log_2 m$
- height = order k
- delete root ⇒ k binom. t. $O(\log_2 m)$ DELETE (ROOT)
- two trees with the same order k can be merged into a binomial tree of order k+1 by setting one to be the LEFTMOST child of the other

sorted LL

BINARY HEAP

- single + complete BT.
- every level is filled, except maybe the last level.
- it merges inefficiently
- a BT having a heap structure and a heap property. (min/max)
- $O(\log_2 m)$ ADD
- $\Theta(1)$ get ROOT
- $O(\log_2 m)$ DELETE (ROOT)

AVL TREE

- BT + extra prop: height(left) - height(right) ∈ {-1, 0, 1} (BST)
- AVLNode:
- info: TComp
left: TAVLNode
right: TAVLNode
h: Integer

init(bt)

bt.root ← pop(s)

buildTree ← bt

AVLTree

root: TAVLNode (relation) → sorting alg.

anc. order ⇒ build a min. heap and remove elems one by one $O(m \log_2 m)$

BinHeap

capacity: Int
mElem: Int
elems: TElem[]

BINOMIAL HEAP

- a collection of binomial trees
- it merges two heaps efficiently → at most $\log_2 m$ Binom. Tr.
- order k ⇒ 2^k nodes; $n = 14 = 1110 = 2^1 + 2^2 + 2^3$ ⇒ BT of orders 1, 2, 3.
- at most one binomial tree of each degree (BFS)
- TOP view → columns → first node found; queue; map
- ANCESTORS of degree k ⇒ $O(\min(m, 2^k))$ = PRINT LEVEL k
- HEIGHT of a tree: using DFS traversal
- ROOT HAS DEGREE 0

ADT BinaryTree

Node

info: TElem
left: TNode
right: TNode

BT:

root: TNode

Op: init(s), push(s, e), pop(s, e)

isEmpty(s), top(s) → stack / queue operations

DT = Domain + Interface ^{(val.) (op)} ^{set of values (dom.) with a set of op.} DSA reference sheet

ADT = no implementation = DT with properties

CONTAINER = collection of data, ± types. which allows accessing elements through an iterator.

SUBALGORITHM = subprogram that does not return a value.

FUNCTION = subprogram that returns a value.

COMPLEXITY ^{abstraction = separation of domain and interface from implementation}

Complexity = how do the nr. of steps change if m increases/...

Nr. of steps = depends on size. (T(m))

O - notation = upper bound

ex: $T(m) = m^2 + 2m + 2$
 $\Rightarrow O(m^2), O(m^3) \checkmark$

Ω - notation = lower bound

Θ - notation (exact)

AC, BC, WC. (first occurrence of a nr. in an array)

↓ min. ↓ max. nr. steps

$\sum_{I \in D} P(I) \cdot E(I) = p_b * nr_op$

BC = WC = AC $\Rightarrow$ TC = Θ

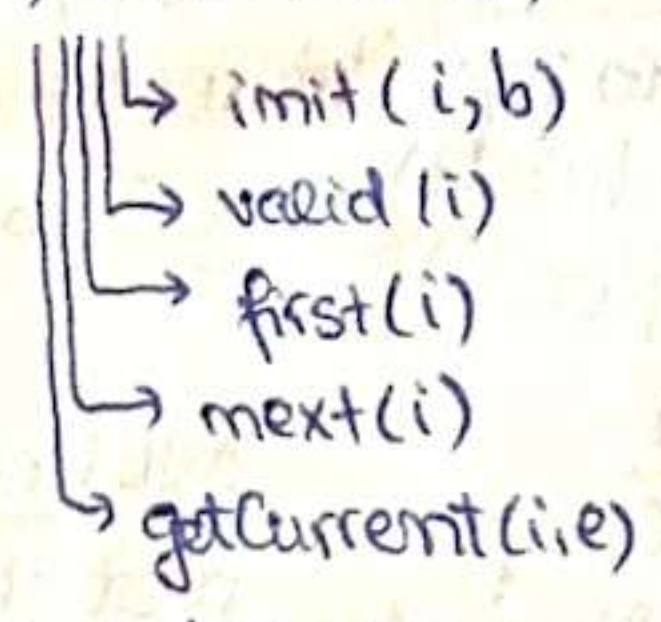
BC $\neq$ WC $\neq$ AC $\Rightarrow$ TC = WC $\Rightarrow$ O

ADT BAG

(shopping cart)

- order does not matter
- uniqueness does not matter
- no positions
- operations:

- 1) init(b)
- 2) add(b, e)
- 3) remove(b, e)
- 4) search(b, e)
- 5) size(b)
- 6) nrOccurrences(b, e)
- 7) destroy(b)
- 8) iterator(b, i)



representations:

- ✓ 1) LIST (dynamic array)
 - 2) 2 LISTS (elem + freq.)
- or

1 LIST of pairs / BDMap

ADT Sequence

-
-
-
-
- operations: $\Theta(1)$

- 1) Empty(sequence)
- 2) init(sequence)
- 3) removeFirst(sequence)
- 4) destroy(sequence)

4

BDHNode
 K: TKey
 V: TValue
 nextKey, prevKey, nextValue, prevValue: BDHNode

BDH: KeyTable, valueTable (BDHNode)

m: Int
 h-key, h-value: TFunction
 size: Int

5) addToEnd(sequence, (K, v))

AMORTIZED = total cost / nr. operations.

(ex: addToEnd in DArray is $\Theta(1)$ amortized because $\Theta(m)$ rarely happens, when resizing occurs.)

BinaryTree Iterator

bt: BinaryTree or queue (nodes)
 stack: Stack (Node)
 current: Node
 STRUCT (static DS) = RECORD

static
 ARRAY (DS)

- orders matters (contiguous mem.)
- fixed nr. of elements (capacity)
- same type of elements (type)
- access in $\Theta(1)$ (+)
- static (-)

ADT Bidirectional map

BDHNode

K: TKey
 V: TValue
 nextKey: BDHNode
 prevKey: BDHNode
 nextValue: BDHNode
 prevValue: BDHNode

BDMap

m: Integer
 keyTable: BDHNode
 valueTable: BDHNode
 hKey: TFunction
 hValue: TFunction
 size: Integer

- init(bdm) $\rightarrow \Theta(1)$
- insert(bdm, k, v) $\rightarrow \Theta(1)$, amortized
- search(bdm, k) $\rightarrow \Theta(1)$, avg
- reverseSearch(bdm, v) $\rightarrow \Theta(1)$
- remove(bdm, k) $\rightarrow \Theta(1)$

PYTHON IMPLEMENTATION (1)

```

class Bag:
    def __init__(self):
        self.__bag = list()
    def add(self, e):
        self.__bag.append(e)
    def remove(self, e):
        if e in self.__bag:
            self.__bag.remove(e)
            return True
        return False
    def search(self, e):
        return e in self.__bag
    def size(self):
        return len(self.__bag)
    def nrOfOccurrences(self, e):
        count = 0
        for elem in self.__bag:
            if elem == e:
                count = count + 1
        return count
    def iterator(self):
        return BagIterator(self)
  
```

class BagIterator:

```

def __init__(self, b):
    self.__bag = b
    self.__current = 0
def valid(self):
    return self.__current < self.__bag.size()
def first(self):
    self.__current = 0
def next(self):
    self.__current += 1
def getCurrent(self):
    if (self.valid()):
        return self.__bag[self.__current]
    raise ValueError()
  
```

ADT Sorted MultiMap

Representation:

TElem:
 K: TKey
 VL: List

SMHNode
 info: TElem
 next: TSMHNode

SMHIterator
 cNode: TSMHNode
 value: ListIterator
 smm: SMH

- Operations:
- init
 - valid
 - next
 - getCurrent
- simply linked
 repres. with
 dynamic alloc.

ADT Sorted Map

Representation:

Node:

K: TKey
 next: Node
 next-sorted: TNode
 prev-sorted: TNode

SM:

m: Integer
 T: (TNode)[]
 K: TFunction
 rel: Relation

Operations:

subalgorithm insertFirst(slla, elem) is:

```

if slla.firstEmpty = -1 then
    newElements ← @ array ... *2 pos
    newNext ← @ array ... *2 pos
    for i ← 1, slla.cap execute
        newElements[i] ← slla.elements[i]
        newNext[i] ← slla.next[i]
    for i ← slla.cap + 1, slla.cap * 2 - 1 ex
        newNext[i] ← i + 1
    newNext[slla.cap * 2] ← -1
    // free slla.next, slla.elements (opt)
    slla.elements ← newElements
    slla.next ← newNext
    slla.firstEmpty ← slla.cap + 1
    slla.cap ← slla.cap * 2
    newPosition ← slla.firstEmpty
    slla.elements[newPosition] ← elem
    slla.firstEmpty ← slla.next[slla.firstEmpty]
    slla.next[newPosition] ← slla.head
    slla.head ← newPosition
    Θ(1) amortized

```

subalg deleteElem(slla, e):

```

prev ← -1
current ← slla.head
while current ≠ -1 and
    slla.elements[current] ≠ e execute
    prev ← current
    current ← slla.next[current]
if current ≠ -1 then
    deleteElem ← false
else
    if prev = -1 then
        slla.head ← slla.next[slla.head]
    else
        slla.next[prev] ← slla.next[current]
    slla.next[current] ← slla.firstEmpty
    slla.firstEmpty ← current
    deleteElem ← true

```

HASH TABLES DO NOT MAINTAIN POSITIONS / SORTED RELATIONS
 ⇒ SET, BAG, MAP, MULTIMAP
 can be represented using HT.
 (order does not matter in these containers) Θ(1) usually ^{insert} _{search}
 cuckoo hashing T₁, T₂; send one to another.

REMOVE = the only operation on HT that cannot be efficiently implemented (searching is expensive)
 Θ(n) (with open addressing)
 (sep. chaining is fine)
 14% of 14 = 14

subalgorithm insertPos(dlla, e, pos):

```

if pos < 1 or pos > dlla.size + 1 then
    @ throw exception
occupiedPos ← allocate(dlla);
if occupiedPos = -1 then
    newArray ← @ new array(*2)
    for i ← 1 to dlla.cap execute
        newArray[i] ← dlla.nodes[i]
    for i ← dlla.cap + 1 to dlla.cap * 2 ex
        newArray[i].next ← i + 1
        newArray[i].prev ← i - 1
    newArray[dlla.cap * 2].next ← -1
    newArray[dlla.cap + 1].prev ← -1
    dlla.firstEmpty ← dlla.cap + 1
    dlla.cap ← dlla.cap * 2
    dlla.nodes ← newArray
    occupiedPos ← allocate(dlla)
dlla.nodes[occupiedPos].info ← e
if pos = 1 then // insert at beginning
    dlla.nodes[occupiedPos].next ← dlla.head
    if dlla.head = -1 then // empty list
        dlla.head ← occupiedPos
        dlla.tail ← occupiedPos
    else // normal list
        dlla.nodes[dlla.head].prev ← occupiedPos
        dlla.head ← occupiedPos
else if pos = dlla.size + 1 then // insert at the end
    dlla.nodes[occupiedPos].prev ← dlla.tail
    dlla.nodes[dlla.tail].next ← occupiedPos
    dlla.tail ← occupiedPos
else // normal insertion (middle)
    currentN ← dlla.head
    currentP ← 1
    while currentP < pos - 1 (BEFORE)
        currentN ← dlla.nodes[currentN].next
        currentP ← currentP + 1
    dlla.nodes[occupiedPos].prev ← currentN
    dlla.nodes[occupiedPos].next ← dlla.nodes[currentN].next
    dlla.nodes[dlla.nodes[currentN].next].prev ← occupiedPos
    dlla.nodes[currentN].next ← occupiedPos
    dlla.size ← dlla.size + 1

```

subalgorithm printContainer(c) is:

```

iterator(c, it)
while valid(it) execute
    elem ← getcurrent(it)
    print elem
    next(it)

```

for efficient removal

Perfect hashing ⇒ multiple hash functions ⇒ use universal hashing
 $H: \{H_{a,b}(x) = (a * x + b) \% p\} \% m \Rightarrow$ hash table of hash tables

HASH TABLES (Access table generalis.)

- for DAT → Θ(1), but 24B and memory iss. (faster, but more memory) → T[k]
- HT → slightly slower, but less memory
 ↳ T[h(k)]
- hash functions should min. collisions

I SEPARATE CHAINING (U on a slot)

Node: (α > 1) HT:
 key: TKey ^{can!} T: ↑ Node[]
 next: ↑ Node m: Integer
 • search: O(α + 1) h: TFunction
 • insert: Θ(1) ± search ...
 Θ(1) = time needed to compute hash fct
 α = average time needed to search one of the m lists, wc: Θ(m) when all are in a slot

HTIterator

ht: HT
 currentPos: Int
 currentNode: ↑ Node

II COALESCE CHAINING (in slot, has)

HT: (α ≤ 1) • insert: Θ(1) Ac, Θ(n) wc
 T: TKey[]
 next: Integer[] • remove Θ(1), Ac: Θ(1)
 m: Integer
 firstEmpty: Integer
 h: TFunction
 HTIterator:
 ht: HT
 current: Integer

III OPEN ADDRESSING (α ≤ 1)

(no pointer, no next)
 • probe sequence
 Linear probing: $h(k, i) = (h'(k) + i) \% m$
 Quadratic probing: $h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \% m$
 Double hashing: $h(k, i) = (h'(k) + i \cdot h''(k)) \% m$

HT:
 T: TKey[] α = α = Θ(1)
 m: Integer wc: Θ(n)
 h: TFunction

dlla.nodes[occupiedPos].prev ← currentN
 dlla.nodes[occupiedPos].next ← dlla.nodes[currentN].next
 dlla.nodes[dlla.nodes[currentN].next].prev ← occupiedPos
 dlla.nodes[currentN].next ← occupiedPos

dlla.size ← dlla.size + 1

LINKED HASH TABLE (DS) • predictable iteration order (= order of insertion)

Node: info: TKey
 nextH: ↑ Node
 nextL: ↑ Node
 prevL: ↑ Node
 Linked HT:
 w: Integer
 T: (↑ Node)[]
 h: TFunction
 tail: ↑ Node, head: ↑ Node
 (if you need in sorted order store Relation)

PERFECT HASHING ⇒ multiple hash functions ⇒ use universal hashing

5

XOR Linked List

- memory-efficient
- DLL where every node retains one link (XOR between the previous and next node)

XOR Node:

info: TElem

link: ↑ Integer // XOR

XOR List:

head: ↑ XORNode

tail: ↑ XORNode

Traversal:

subalgorithm

Skip List

- store sorted elems
- insertion has good complexity for skip list
- insert = find + actual insertion
- search: $O(\log_2 m)$
- probabilistic DS (height)

SEARCH/CONTAINER

DA: $\Theta(m)$

LL: $\Theta(m)$

BST: $\Theta(\log m)$

AVL: $\Theta(\log m)$

HT: $\Theta(1)$

BH: $\Theta(m)$

(binary)

Linked Lists on Arrays

→ if we work in a programming language with no pointers. (use arrays instead)

	1	2	3	4	5	6	
elems:	46	78	11	6	59	19	...
next:	5	6	1	-1	2	4	...

!head = 3

⇒ order: 11, 46, 59, 78, 19, 6

• insert 44: (3rd position)

	1	2	3	4	5	6	7	8
elems:	46	78	11	6	59	19	44	
next:		6	5	-1	8	4	2	

!head = 3

⇒ order: 11, 59, 44, 78, 19, 6

• linked list of empty positions!

SLL

current ← sll.head

current ← [current].next

- top of the stack should be at the beginning of the list

Queue on SLL

Node:

info: TElem

next: ↑ Node

Queue:

head: Node

tail: Node

nrElem: Int

- Store tail because otherwise push would require traversing the whole list.

front(pop) rear(push) ⇒ good if we pop (BETTER THAN !)

rear(push) front(pop) ⇒ good if we push

Priority Queue on DLL

PQElem:

elem: TElem

priority: TPriority

Node

info: PQElem

next: Node

- does not matter where the top of the stack is (in the list)

Queue on DLL

Node:

info: TElem

prev: Node

next: Node

Queue:

head: Node

tail: Node

nrElem: Int

- does not matter where we put front/rear
- does not improve Queue op.

ADT Deque on DLL

Node:

info: TElem

prev: Node

next: Node

Deque

head: Node

tail: Node

nrElem: Int

DLLANode:

(dynamically allocated)

info: TElem

next: Integer

prev: Integer

DLLA

nodes: DLLANode[]

cap: Integer

head: Integer

tail: Integer

firstEmpty: Integer

(size: Integer)

current ← dlla.head

current ← dlla.nodes[current].next

function allocate(dlla) is:

subalgorithm free(dlla, pos) is:

subalgorithm InsertPosition(dlla, elem, pos)

DLLA Iterator

$\Theta(1)$

list: DLLA

currentElement: Integer

subalgorithm init(it, dlla) is:

it.list ← dlla $\Theta(1)$

it.currentElement ← dlla.head

subalgorithm getCurrent(it) is:

if it.currentElement = -1 then

@throw-exception $\Theta(1)$

getCurrent ← it.list.nodes[it.

currentElement].info

subalgorithm next(it)

function valid(it)

SLLA:

elems: TElem[]

next: Integer[]

cap: Integer

head: Integer

firstEmpty: Integer

function search(slla, elem) is:

current ← slla.head; index of head

while current ≠ -1 and

slla.elems[current] ≠ elem ex

current ← slla.next[current]

end-while; stop traversal when

if current ≠ -1 then current = -1

search ← True $O(m)$

else

search ← False

end-if

end-function

subalgorithm init(slla) is:

slla.cap ← INIT_CAPACITY

slla.elems ← @ array w slla.cap

slla.next ← @ array ... pos

slla.head ← -1

for i ← 1, slla.cap - 1 ex

slla.next[i] ← i + 1

slla.next[slla.cap] ← -1

slla.firstEmpty ← 1

subalgorithm insertFirst(slla, elem)

subalgorithm insertPosition(slla, elem, pos)

MERGE SORTED LINKED-LISTS (SLL)

L1: $\boxed{3} \rightarrow \boxed{5} \rightarrow \boxed{9} \rightarrow \boxed{10} \rightarrow \boxed{11} \rightarrow \boxed{X}$ (sorted)

L2: $\boxed{1} \rightarrow \boxed{2} \rightarrow \boxed{6} \rightarrow \boxed{7} \rightarrow \boxed{X}$ (sorted)

LR: $\boxed{1} \rightarrow \boxed{2} \rightarrow \boxed{3} \rightarrow \boxed{5} \rightarrow \boxed{6} \rightarrow \boxed{7} \rightarrow \dots$

• you can just change the links between the nodes if you don't need L1, L2.

• otherwise, take LR (result list)

• Representation:

Node:

value: TComp

next: $\uparrow$ Node (pointer)

List:

head: $\uparrow$ Node

(R: Relation)

• Implementation:

subalgorithm merge(L1, L2, LR)

if not L1.R([L1.head].value, [L2.head].value) then

tmp $\leftarrow$ L1.head

L1.head $\leftarrow$ L2.head

L2.head $\leftarrow$ tmp

L1.head $\leftarrow$ L1.head (guaranteed the "smaller")

CL1 $\leftarrow$ [L1.head].next (second node of L1)

CL2 $\leftarrow$ L2.head (current first node of L2)

while CL2 $\neq$ Nil and CL1 $\neq$ Nil ex: $\checkmark$

if L1.R([CL1].value, [CL2].value) then

chosen $\leftarrow$ CL1; choose "smaller" (from L1)

CL1 $\leftarrow$ [CL1].next; advance

else

chosen $\leftarrow$ CL2

CL2 $\leftarrow$ [CL2].next

LRtail $\leftarrow$ L1.head (initially)

[LRtail].next $\leftarrow$ chosen (attach chosen)

LRtail $\leftarrow$ chosen; move tail pointer
(chosen is after tail)

if CL1 $\neq$ Nil; L1 still has nodes left

[LRtail].next $\leftarrow$ CL1; attach directly bc L1
is already sorted

else

[LRtail].next $\leftarrow$ CL2

LR.R $\leftarrow$ L1.R; copy the ordering relation

L1.head $\leftarrow$ Nil; empty L1 & L2.

L2.head $\leftarrow$ Nil

MOST IMPORTANT DS:

I. DYNAMIC ARRAY (random access)

$\rightarrow$ contiguous memory; fast indexing

$\rightarrow$ insert/delete is expensive in the middle

• operations: $\Theta(1)$: access by index, delete end, insert end $\Theta(1)$ amortized;

• op: $\Theta(n)$: search, insert/delete middle

II. SLL (many insertions/removals)

$\rightarrow$ fast insertion/deletion

$\rightarrow$ no positions

$\rightarrow$ op: $\Theta(1)$: insert/delete front

$\rightarrow$ op: $\Theta(n)$: search, access pos p

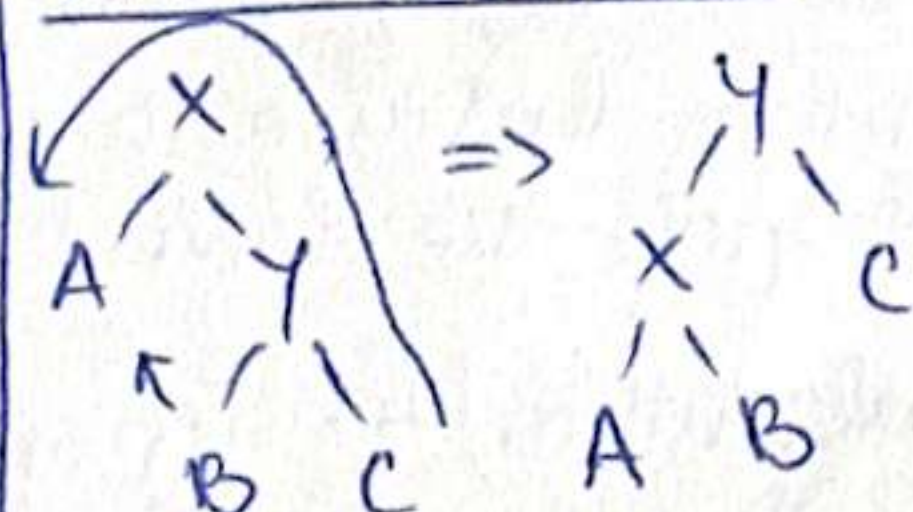
III. DLL

$\rightarrow$ fast deletion

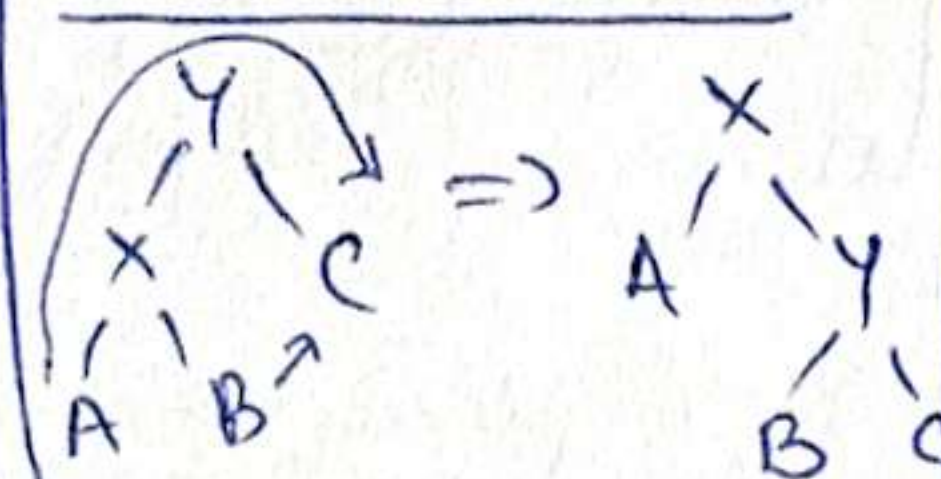
$\rightarrow$ more memory

IV.

LEFT ROTATION:



RIGHT ROTATION:



Bucket Sort (STABLE S.A.)

- sort a set of pairs based on the keys.
- if we have equal keys, they will appear in the order they were added.

S: (4,d), (1,c), (3,b), (7,g), (3,a), (7,c)

B:

0	1	2	3	4	5	6	7
∅	∅	∅	∅	∅	∅	∅	∅

 (1,c) (3,b), (3,a) (7,d), (7,g), (7,c)

S: (1,c), (3,b), (3,a), (7,d), (7,g), (7,c)

- circular array (DS) if we wanted $\Theta(1)$. (for all operations)

ADT Sequence ~ Queue

$\Theta(1)$ {
 init(s)
 removeFirst(s)
 addToEnd(s, (k,v))
 destroy(s)
 isEmpty(s)

Subalgorithm BucketSort(S, N)

@define array B of dimension N
 while not isEmpty(s) execute: $\Theta(m)$
 (k,v) ← removeFirst(s)
 addToEnd(B[k], (k,v))
 for i=0, N execute: $\Theta(m+N)$
 while not isEmpty(B[i]) ex
 (k,v) ← removeFirst(B[i])
 addToEnd(S, (k,v))

DS Iterator

data: DS
 current: TElem / Node

- init
- first
- getCurrent
- valid
- next

m = nr of pairs
 N = dimension of B
 → keys = integers!

Lexicographic Sort

- sort a set of d-pairs
- compare the first dimension, if they're equal → the second etc.

R_i = relation comparing 2 tuples considering the i^{th} position

$(x_1, x_2, x_3, \dots, x_d)$
 $(y_1, y_2, y_3, \dots, y_d)$

- stableSort(S, R) - stable sorting algorithm that uses a relation R to compare the elements. (d times)

Subalgorithm LexicographicSort(S, d, R)

for $i \leftarrow d, 1, -1$ execute $\Theta(d \cdot T(m))$
 stableSort(S, R_i)

$T(m)$ = complexity of stableSort

S: (7,4,6), (5,1,5), (2,4,6), (2,1,4), (3,2,4)

d=3: (2,1,4), (3,2,4), (5,1,5), (7,4,6), (2,4,6)

d=2: (2,1,4), (5,1,5), (3,2,4), (7,4,6), (2,4,6)

d=1: (2,1,4), (2,4,6), (3,2,4), (5,1,5), (7,4,6)

Radix Sort (STABLE S.A. = BucketSort)

- LexicographicSort + BucketSort

$\Theta(d \cdot (N+m)) \approx \Theta(m)$

↳ how many nr. we are sorting
 ↳ nr. of buckets = nr. digits = 10
 ↳ nr. of digits in the max. nr.

(2,4,6) ⇒ 246 } You need the same length!
 (0,3,5) ⇒ 35

- linear complexity because it never compares those tuples directly

12) ADT Priority Queue → HPT (highest priority first) → each elem. has an associated priority! → NO ITERATOR

13) ADT Deque (double-ended queue) → insert/delete from both ends

14) ADT List → if TPosition = int ⇒ IndexedList
 ↳ if TPosition = iterator ⇒ IteratedList

15) ADT SortedList → indexed/iterated

ADTs

- 1) ADT Bag → not unique, no pos, no order.
- 2) ADT SortedBag → no pos, yes order (+ relation)
- 3) ADT Set → unique, no pos, no order
- 4) ADT SortedSet → set + relation
- 5) ADT Matrix → two-dimensional array → sparse matrix if many values of 0 → memorize only $\neq 0$ val.
- 6) ADT Map → no pos, no order, not necessarily unique values, but keys must be unique, store (key, value) pairs
- 7) ADT SortedMap → map with sorted keys (relation)
- 8) ADT MultiMap → map where a key can have multiple values
- 9) ADT SortedMultiMap → multiMap + ordered keys (values don't matter)
- 10) ADT Stack → LIFO → access restricted to TOP (NO ITERATOR)
- 11) ADT Queue → FIFO → push to REAR, pop to FRONT (if we push more) (NO ITERATOR)